

CIS 3400: Database Management Systems

Spring 2026

20 May 2026

Building Maintenance & Work Order Management System

Group 9

Abdelrahman Imam - iramann16@gmail.com

Bernie Yang - bernieyang2024@gmail.com

Gyuhyouun (Evelyn) Kim - gyuutti33@gmail.com

Madison Dusold - madusold22@gmail.com

Victor Torres - victorres101010@gmail.com

Table of Contents

I . Proposal

- a. Overview
- b. Goals
- c. Information Needed
- d. Initial List of Entities
- e. Duties

II . Systems Analysis and E-R Model

- a. Finalized List of Entities
- b. Relationship Sentences
- c. UML

III. Logical and Physical Modeling

- a. Relational Data Model (RDM)
- b. Functional Dependencies
- c. Normalization

IV. Database Implementation

- a. SQL Table Creation, Constraints, and Sample Data
- b. Database Structure Summary

V. Application Implementation

- a. Business Scenarios, SQL Queries, Query Purpose and Explanation

VI. Conclusion

- a. Project Experience and Reflection
- b. Expected Benefits of the System

I . Proposal

a. Overview

Our group proposes the design and implementation of a Building Maintenance and Work Order Management System for a mid-sized company that operates across multiple office locations and/or multiple floors within a single facility. This organization currently relies on an outdated and decentralized process for handling maintenance requests. Employees typically submit maintenance requests through emails, phone calls, verbal communication, or written notes. As a result, requests are often delayed, lost, or even duplicated. There is no consistent method for tracking each request status. The lack of a structured system has created several operational challenges. Response times are slow because maintenance staff do not receive requests in an organized or prioritized manner. Additionally, there is no reliable way to monitor task progress, leading to poor communication between employees and maintenance personnel. Because of these underlying problems, the organization cannot easily evaluate maintenance performance, track costs, or identify recurring issues. This limits the company's ability to improve efficiency and allocate resources effectively. The proposed system accounts for these issues and presents an opportunity to improve operational efficiency and communication within the organization. The system will also provide valuable insights through reporting and analysis.

b. Goals

The primary goal of this project is to design and implement a normalized relational database system that models the full maintenance work order process. The system aims to achieve the following:

- Multi-location tracking across floors and office sites
- Work order status summaries and technician performance reports
- Maintenance cost breakdowns by location, equipment type, or time period
- Reduced lost or delayed requests through centralized data management
- A historical maintenance record to support future budgeting and asset decisions

c. Information Needed

To improve the efficiency and effectiveness of maintenance operations, the organization requires accurate, timely, and relevant information related to maintenance activities. The proposed system must capture detailed information about each maintenance request, including the problem description, location, date and time submitted, and priority level. It should also store employee information such as name, department, and contact details to ensure clear communication and accountability. In addition, the system must track maintenance staff, including their skills, availability, and assigned work orders. Information about work order progress, such as status updates, start and completion times, and resolution details, is essential for monitoring performance and reducing delays. The system should also collect data on materials used, repair costs, and external vendors to support cost tracking and budgeting. Finally, management requires reporting and performance metrics, such as response times, frequency of recurring issues, and workload distribution, to identify inefficiencies, improve resource allocation, and support data-driven decision-making.

d. Initial List of Entities

The initial entities identified for this system include:

Location, Departments, WorkOrder, WorkOrderStatusHistory, Employee, AssignmentHistory, Part, Vendor, PartUsage, InspectionSchedule

e. Duties

Name	Role
Abdelrahman Imam	Contributed to entity development, normalization, and supported parts of the final report
Bernie Yang	Prepared the RDM and handled SQL table creation, constraints, and sample data
Gyuhyouun (Evelyn) Kim	Served as the project lead and was responsible for the ERD, report organization, database structure, and conclusion sections
Madison Dusold	Focused on the application implementation section, especially the business scenarios
Victor Torres	Contributed to the development of the relationship sentences and supported logical modeling, SQL, and database review

II . Systems Analysis and E-R Model

a. Finalized List of Entities

Location (LocationID, BuildingName, FloorNumber, RoomNumber, SiteName)

Department (DepartmentID, DepartmentName, DepartmentPhone)

Employee (EmployeeID, FirstName, LastName, Email, PhoneNumber, DepartmentID(fk))

Technician (TechnicianID(fk), SkillSet, AvailabilityStatus)

Equipment (EquipmentID, EquipmentName, EquipmentType, InstallDate, LocationID(fk))

WorkOrder (WorkOrderID, ProblemDescription, DateSubmitted, TimeSubmitted, PriorityLevel, ResolutionDetails, RequesterEmployeeID(fk), LocationID(fk), EquipmentID(fk))

WorkOrderStatusHistory (StatusHistoryID, WorkOrderID(fk), Status, StatusDate, StatusTime, Notes)

AssignmentHistory (AssignmentID, WorkOrderID(fk), TechnicianID(fk), AssignedDate, StartDateTime, CompletionDateTime)

Part (PartID, PartName, PartDescription, UnitCost, VendorID(fk))

Vendor (VendorID, VendorName, ContactName, PhoneNumber, Email)

PartUsage (PartUsageID, WorkOrderID(fk), PartID(fk), QuantityUsed)

InspectionSchedule (InspectionID, LocationID(fk), TechnicianID(fk), InspectionDate, InspectionType, InspectionStatus)

b. Relationship Sentences

Department - Employee

A department can have 0 or more employees : 0 : *

Each employee belongs to only 1 department : 1 : 1

Employee - WorkOrder (Requester)

An employee can submit 0 or more work orders : 0 : *

Each work order is submitted by 1 or more employees : 1 : *

Location - Equipment

A location can have 0 or more equipment : 0 : *

Each equipment is located at only 1 location : 1 : 1

Location - WorkOrder

A location can have 0 or more work orders : 0 : *

Each work order is associated with only 1 location : 1 : 1

Equipment - WorkOrder

An equipment can be referenced in 0 or more work orders : 0 : *

Each work order may reference 0 or 1 equipment : 0 : 1

WorkOrder - WorkOrderStatusHistory

A work order has 1 or more status history records : 1 : *

Each status history record belongs to only 1 work order : 1 : 1

WorkOrder - AssignmentHistory

A work order can have 0 or more assignments : 0 : *

Each assignment belongs to only 1 work order : 1 : 1

Technician - AssignmentHistory

A technician can have 0 or more assignments : 0 : *

Each assignment involves 1 or more technician : 1 : *

WorkOrder - PartUsage

A work order can have 0 or more part usage records : 0 : *

Each part usage record belongs to only 1 work order : 1 : 1

Part - PartUsage

A part can appear in 0 or more part usage records : 0 : *

Each part usage record references only 1 part : 1 : 1

Vendor - Part

A vendor can supply 0 or more parts : 0 : *

Each part is supplied by only 1 vendor : 1 : 1

Location - InspectionSchedule

A location can have 0 or more inspection schedules : 0 : *

Each inspection schedule is for only 1 location : 1 : 1

Technician - InspectionSchedule

A technician can perform 0 or more inspections : 0 : *

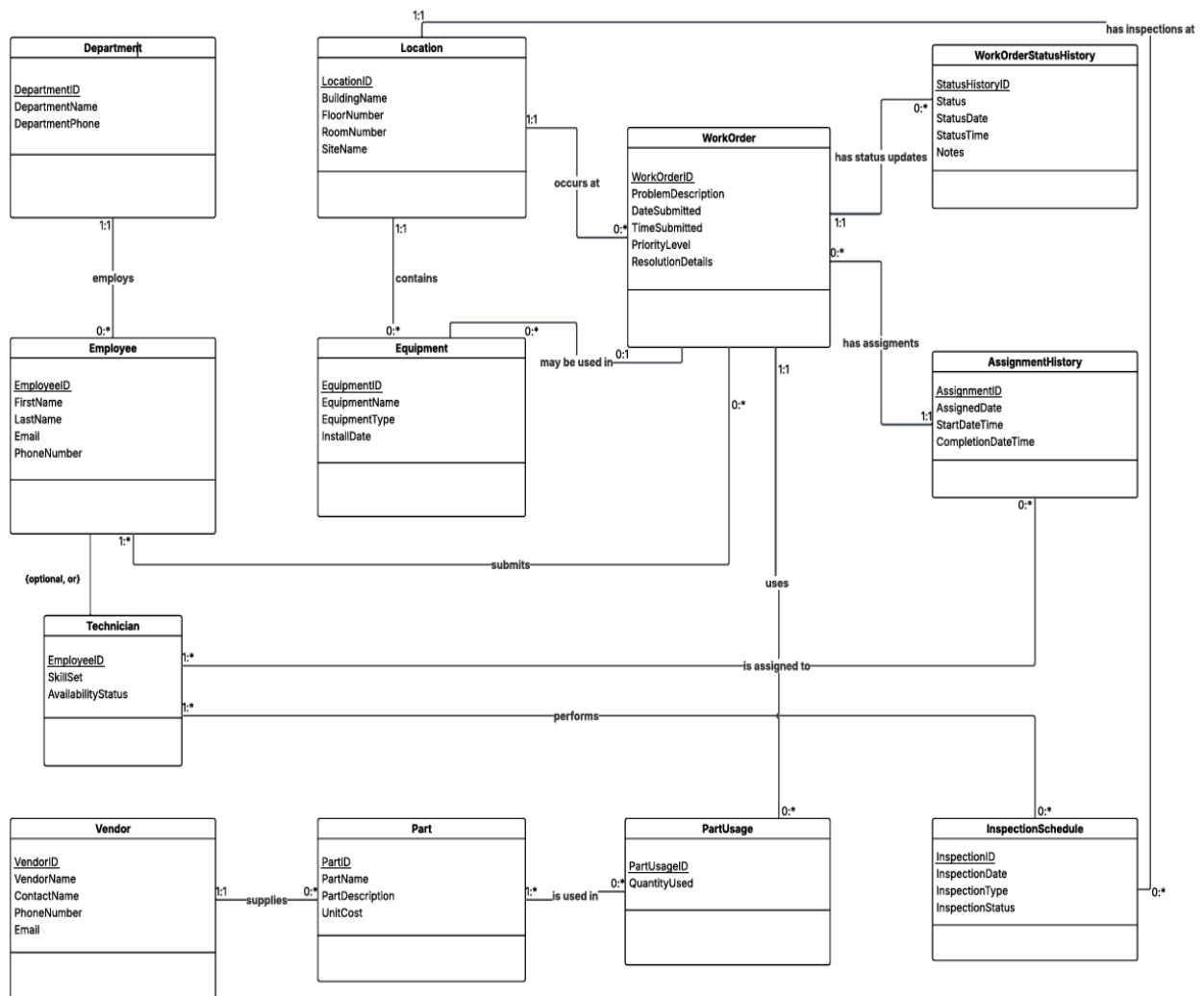
Each inspection schedule is assigned to 1 or more technician : 1 : *

Employee - Technician (subtype)

Employee is the supertype, and Technician is a subtype of Employee.

Participation is optional because not every employee is a technician.

c. UML



III. Logical and Physical Modeling

a. RDM

Department(DepartmentID, ...)

Employee(EmployeeID, ..., DepartmentID(fk))

Technician(EmployeeID(fk), ...)

Employee_WorkOrder(EmployeeID(fk), WorkOrderID(fk))

WorkOrder(WorkOrderID, ..., LocationID(fk), AssignmentID(fk))

Location(LocationID, ...)

Equipment(EquipmentID, ..., LocationID(fk), WorkOrderID(fk))

WorkOrderStatusHistory(StatusHistoryID, ..., WorkOrderID(fk))

InspectionSchedule(InspectionID, ..., LocationID(fk))

AssignmentHistory(AssignmentID, ...)

Technician_AssignmentHistory(EmployeeID(fk), AssignmentID(fk))

Technician_InspectionSchedule(EmployeeID(fk), InspectionID(fk))

PartUsage(PartUsageID, ..., WorkOrderID(fk))

Part(PartID, ..., VendorID(fk))

Part_PartUsage(PartID(fk), PartUsageID(fk))

Vendor(VendorID, ...)

b. Normalization

Step 1: Key and functional dependencies 1NF

Department

Key: DepartmentID -> 1NF

FD1: DepartmentID -> DepartmentName, DepartmentPhone

Partial dependency? No -> 2NF

Transitive dependency? No -> 3NF

Employee

Key: EmployeeID -> 1NF

FD2: EmployeeID -> FirstName, LastName, Email, PhoneNumber, DepartmentID

Partial dependency? No -> 2NF

Transitive dependency? No -> 3NF

Technician

Key: EmployeeID -> 1NF

FD3: EmployeeID -> SkillSet, AvailabilityStatus

Partial dependency? No -> 2NF

Transitive dependency? No -> 3NF

WorkOrder

Key: WorkOrderID -> 1NF

FD4: WorkOrderID -> ProblemDescription, DateSubmitted, TimeSubmitted, PriorityLevel, ResolutionDetails, LocationID, AssignmentID

Partial dependency? No -> 2NF

Transitive dependency? No -> 3NF

Location

Key: LocationID -> 1NF

FD5: LocationID -> BuildingName, FloorNumber, RoomNumber, SiteName

Partial dependency? No -> 2NF

Transitive dependency? No -> 3NF

Equipment

Key: EquipmentID -> 1NF

FD6: EquipmentID -> EquipmentName, EquipmentType, InstallDate, LocationID, WorkOrderID

Partial dependency? No -> 2NF

Transitive dependency? No -> 3NF

WorkOrderStatusHistory

Key: StatusHistoryID -> 1NF

FD7: StatusHistoryID -> WorkOrderID, Status, StatusDate, StatusTime, Notes

Partial dependency? No -> 2NF

Transitive dependency? No -> 3NF

InspectionSchedule

Key: InspectionID -> 1NF

FD8: InspectionID -> LocationID, InspectionDate, InspectionType, InspectionStatus

Partial dependency? No -> 2NF

Transitive dependency? No -> 3NF

AssignmentHistory

Key: AssignmentID -> 1NF

FD9: AssignmentID -> WorkOrderID, AssignedDate, StartDateTime, CompletionDateTime

Partial dependency? No -> 2NF

Transitive dependency? No -> 3NF

PartUsage

Key: PartUsageID -> 1NF

FD10: PartUsageID -> WorkOrderID, PartID, QuantityUsed

Partial dependency? No -> 2NF

Transitive dependency? No -> 3NF

Part

Key: PartID -> 1NF

FD11: PartID -> PartName, PartDescription, UnitCost, VendorID

Partial dependency? No -> 2NF

Transitive dependency? No -> 3NF

Vendor

Key: VendorID -> 1NF

FD12: VendorID -> VendorName, ContactName, PhoneNumber, Email

Partial dependency? No -> 2NF

Transitive dependency? No -> 3NF

12 relations all in 3NF: Department, Employee, Technician, WorkOrder, Location, Equipment, WorkOrderStatusHistory, InspectionSchedule, AssignmentHistory, PartUsage, Part, Vendor.

IV. Database Implementation

a. SQL Table Creation, Constraints, and Sample Data

A summary of the database structure is included on page 19.

```
-- Departments
```

```
CREATE TABLE Departments(  
    department_id VARCHAR(100) NOT NULL,  
    department_name VARCHAR(100) NOT NULL,  
    department_phone VARCHAR(20)  
);
```

```
ALTER TABLE Departments  
ADD CONSTRAINT DEPARTMENTS_PK PRIMARY KEY(department_id)
```

```
-- Employees
```

```
CREATE TABLE Employees(  
    employee_id VARCHAR(100) NOT NULL,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    email VARCHAR(100) NOT NULL,  
    phone VARCHAR(20),  
    department_id VARCHAR(100),  
    location_id VARCHAR(100) NOT NULL  
);
```

```
ALTER TABLE Employees  
ADD CONSTRAINT EMPLOYEE_PK PRIMARY KEY(employee_id)
```

```
ALTER TABLE Employees  
ADD CONSTRAINT EMPLOYEE_FK FOREIGN KEY(department_id) REFERENCES  
Departments
```

```
-- Technician Employees
CREATE TABLE Technicians(
    employee_id VARCHAR(100) NOT NULL,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100),
    phone VARCHAR(20),
    skill_set VARCHAR(255),
    availability_status VARCHAR(50),
    hire_date DATETIME
);

ALTER TABLE Technicians
ADD CONSTRAINT TECHNICIANS_PK PRIMARY KEY(employee_id)

ALTER TABLE Technicians
ADD CONSTRAINT TECHNICIANS_FK FOREIGN KEY(employee_id) REFERENCES
Employees(employee_id)

-- Joint table to show the relationship of employee + workorder
CREATE TABLE Employee_WorkOrder(
    employee_id VARCHAR(100) NOT NULL,
    work_order_id VARCHAR(100) NOT NULL
);

ALTER TABLE Employee_WorkOrder
ADD CONSTRAINT EMPLOYEE_WORKORDER_PK PRIMARY KEY(employee_id,
work_order_id)

ALTER TABLE Employee_WorkOrder
ADD CONSTRAINT EMPLOYEE_WORKORDER_EMPLOYEE_FK FOREIGN
KEY(employee_id) REFERENCES Employees

ALTER TABLE Employee_WorkOrder
ADD CONSTRAINT EMPLOYEE_WORKORDER_WORKORDER_FK FOREIGN
KEY(work_order_id) REFERENCES WorkOrders
```

-- Work orders

```
CREATE TABLE WorkOrders(  
    work_order_id VARCHAR(100) NOT NULL,  
    work_order_description VARCHAR(100),  
    work_order_timestamp DATETIME,  
    resolution_details VARCHAR(100),  
    location_id VARCHAR(100) NOT NULL,  
    assignment_id VARCHAR(100) NOT NULL  
);
```

```
ALTER TABLE WorkOrders  
ADD CONSTRAINT WORKORDERS_PK PRIMARY KEY(work_order_id)
```

```
ALTER TABLE WorkOrders  
ADD CONSTRAINT WORKORDERS_LOCATION_FK FOREIGN KEY(location_id)  
REFERENCES Locations
```

```
ALTER TABLE WorkOrders  
ADD CONSTRAINT WORKORDERS_ASSIGNMENT_FK FOREIGN KEY(assignment_id)  
REFERENCES Assignments
```

-- Locations

```
CREATE TABLE Locations(  
    location_id VARCHAR(100) NOT NULL,  
    building_name VARCHAR(100) NOT NULL,  
    floor_number VARCHAR(20),  
    room_number VARCHAR(20),  
    address VARCHAR(255)  
);
```

```
ALTER TABLE Locations  
ADD CONSTRAINT LOCATIONS_PK PRIMARY KEY (location_id)
```

```
-- Equipments
CREATE TABLE Equipments(
    equipment_id VARCHAR(100) NOT NULL,
    equipment_name VARCHAR(100) NOT NULL,
    equipment_type VARCHAR(20),
    location_id VARCHAR(100) NOT NULL,
    work_order_id VARCHAR(100) NOT NULL,
    install_date DATETIME
);

ALTER TABLE Equipments
ADD CONSTRAINT EQUIPMENT_PK PRIMARY KEY(equipment_id)

ALTER TABLE Equipments
ADD CONSTRAINT EQUIPMENT_LOCATION_FK FOREIGN KEY(location_id) REFERENCES
Locations

ALTER TABLE Equipments
ADD CONSTRAINT EQUIPMENT_WORKORDER_FK FOREIGN KEY(work_order_id)
REFERENCES WorkOrders

-- Work Order Status History
CREATE TABLE WorkOrderStatusHistory(
    status_id VARCHAR(100) NOT NULL,
    work_order_id VARCHAR(100) NOT NULL,
    status_update VARCHAR(100),
    comments VARCHAR(200),
    status_timestamp DATETIME
);

ALTER TABLE WorkOrderStatusHistory
ADD CONSTRAINT STATUS_PK PRIMARY KEY(status_id)

ALTER TABLE WorkOrderStatusHistory
ADD CONSTRAINT STATUS_FK FOREIGN KEY(work_order_id) REFERENCES WorkOrders
```

```
-- Inspection
CREATE TABLE Inspections(
    inspection_id VARCHAR(100) NOT NULL,
    location_id VARCHAR(100) NOT NULL,
    inspection_date DATETIME,
    inspection_type VARCHAR(100),
    inspection_status VARCHAR(100)
);

ALTER TABLE Inspections
ADD CONSTRAINT INSPECTION_PK PRIMARY KEY(inspection_id)
ALTER TABLE Inspections
ADD CONSTRAINT INSPECTION_FK FOREIGN KEY(location_id) REFERENCES Locations

-- Joint table to show the relationship of technician + assignment
CREATE TABLE Technician_Assignment(
    employee_id VARCHAR(100) NOT NULL,
    assignment_id VARCHAR(100) NOT NULL
);

ALTER TABLE Technician_Assignment
ADD CONSTRAINT TECHNICIAN_ASSIGNMENT_PK PRIMARY KEY(employee_id,
assignment_id)

ALTER TABLE Technician_Assignment
ADD CONSTRAINT TECHNICIAN_ASSIGNMENT_EMPLOYEE_FK FOREIGN
KEY(employee_id) REFERENCES Technicians

ALTER TABLE Technician_Assignment
ADD CONSTRAINT TECHNICIAN_ASSIGNMENT_ASSIGNMENT_FK FOREIGN
KEY(assignment_id) REFERENCES Assignments
```

```
-- Assignment
CREATE TABLE Assignments(
    assignment_id VARCHAR(100) NOT NULL,
    work_order_id VARCHAR(100) NOT NULL,
    assignment_date DATETIME,
    completion_date DATETIME
);

ALTER TABLE Assignments
ADD CONSTRAINT ASSIGNMENT_PK PRIMARY KEY(assignment_id)

-- Joint table to show the relationship of technician + inspection
CREATE TABLE Technician_Inspection(
    employee_id VARCHAR(100) NOT NULL,
    inspection_id VARCHAR(100) NOT NULL
);

ALTER TABLE Technician_Inspection
ADD CONSTRAINT TECHNICIAN_INSPECTION_PK PRIMARY KEY(employee_id,
inspection_id)

ALTER TABLE Technician_Inspection
ADD CONSTRAINT TECHNICIAN_INSPECTION_EMPLOYEE_FK FOREIGN
KEY(employee_id) REFERENCES Technicians

ALTER TABLE Technician_Inspection
ADD CONSTRAINT TECHNICIAN_INSPECTION_INSPECTION_FK FOREIGN
KEY(inspection_id) REFERENCES Inspections
```

```
-- Part
CREATE TABLE Parts(
    part_id VARCHAR(100) NOT NULL,
    part_name VARCHAR(100) NOT NULL,
    description TEXT,
    unit_cost FLOAT NOT NULL,
    quantity_in_stock VARCHAR(100),
    vendor_id VARCHAR(100) NOT NULL
);

ALTER TABLE Parts
ADD CONSTRAINT PARTS_PK PRIMARY KEY(part_id)

ALTER TABLE Parts
ADD CONSTRAINT PARTS_FK FOREIGN KEY(vendor_id) REFERENCES Vendors

-- Joint table to show the relationship of part + part usage
CREATE TABLE Part_PartUsage(
    part_id VARCHAR(100) NOT NULL,
    part_usage_id VARCHAR(100) NOT NULL
);

ALTER TABLE Part_PartUsage
ADD CONSTRAINT PART_PARTUSAGE_PK PRIMARY KEY(part_id, part_usage_id)
ALTER TABLE Part_PartUsage
ADD CONSTRAINT PARTS_PARTUSAGE_PART_FK FOREIGN KEY(part_id) REFERENCES
Parts

ALTER TABLE Part_PartUsage
ADD CONSTRAINT PARTS_PARTUSAGE_PARTUSAGE_FK FOREIGN KEY(part_usage_id)
REFERENCES PartUsage
```

```
-- Part Usage
CREATE TABLE PartUsage(
    part_usage_id VARCHAR(100) NOT NULL,
    quantity_used VARCHAR(100) NOT NULL,
    total_cost float NOT NULL,
    work_order_id VARCHAR(100) NOT NULL
);

ALTER TABLE PartUsage
ADD CONSTRAINT PARTUSAGE_PK PRIMARY KEY(part_usage_id)

ALTER TABLE PARTUSAGE
ADD CONSTRAINT PARTUSAGE_FK FOREIGN KEY(work_order_id) REFERENCES
WorkOrders

-- Vendors
CREATE TABLE Vendors(
    vendor_id VARCHAR(100) NOT NULL,
    vendor_name VARCHAR(100) NOT NULL,
    contact_person VARCHAR(100),
    phone VARCHAR(20),
    email VARCHAR(100),
    service_type VARCHAR(100)
);

ALTER TABLE Vendors
ADD CONSTRAINT VENDOR_PK PRIMARY KEY(vendor_id)
```

b. Database Structure Summary

The database is mainly centered on work orders and the maintenance tasks related to them. Main tables like Departments, Employees, Technicians, WorkOrders, and Locations store the basic information needed for the system. Other tables, such as Assignments, Inspections, WorkOrderStatusHistory, Parts, PartUsage, and Vendors, are used to keep track of work progress, inspections, status changes, and materials used. There are also junction tables like Employee_WorkOrder, Technician_Assignment, Technician_Inspection, and Part_PartUsage to

handle relationships between tables that involve many-to-many connections. Overall, the tables are linked through primary keys and foreign keys so that the data stays organized and consistent.

V . Application Implementation

a. Business Scenarios, SQL Queries, Query Purpose and Explanation

1 - View All Technicians

Scenario: A supervisor wants to see all technicians currently in the system, including their skill sets and availability, in order to decide who can be assigned to maintenance work.

SQL:
SELECT
 first_name,
 last_name,
 skill_set,
 availability_status
FROM Technicians;

Purpose: This query allows management to view all technicians in the system along with their skills and current availability. It can help when assigning technicians to new work orders.

2 - Track Parts Used for Repairs

Scenario: A manager wants to review which parts were used for each repair and the related cost in order to better monitor maintenance expenses.

SQL:
SELECT
 PartUsage.part_usage_id,
 PartUsage.work_order_id,
 PartUsage.quantity_used,
 PartUsage.total_cost,
 Parts.part_name
FROM PartUsage
INNER JOIN Part_PartUsage
 ON PartUsage.part_usage_id = Part_PartUsage.part_usage_id
INNER JOIN Parts
 ON Part_PartUsage.part_id = Parts.part_id;

Purpose: This query shows the parts used for repairs, how many were used, and the total cost. It helps track material usage and supports cost monitoring.

3 - View Inspection Schedules

Scenario: A supervisor wants to review all scheduled inspections and the locations where they will take place.

SQL:

```
SELECT
    Inspections.inspection_date,
    Inspections.inspection_type,
    Inspections.inspection_status,
    Inspections.location_id
FROM Inspections;
```

Purpose: This query helps management review inspection schedules together with location details. It can be used to monitor upcoming inspections and check their status.

4 - View equipment for work orders

Scenario: The company wants to identify which equipment is connected to each work order so that equipment-related repairs can be tracked more clearly.

SQL:

```
SELECT
    equipment_id,
    equipment_name,
    equipment_type,
    work_order_id
FROM Equipments;
```

Purpose: This query shows the equipment linked to work orders. It can help the company track equipment repairs and review maintenance history for specific items.

5 - View all Work Orders

Scenario: A supervisor wants to review all work orders in the system along with their location information.

SQL:

```
SELECT
    work_order_id,
    work_order_description,
    work_order_timestamp
FROM WorkOrders;
```

Purpose: This query allows users to view all work orders together with their locations. It helps management monitor maintenance requests more clearly and keep track of where each issue occurred.

VI. Conclusion

a. Project Experience and Reflection

This project gave our team a better understanding of how a database system is designed from beginning to end. As we worked through the proposal, ERD, relational model, normalization, and SQL implementation, we realized that each part depended heavily on the accuracy of the previous step. One of the hardest parts was revising our work after feedback and making sure that all sections stayed consistent. However, working as a team made the process easier because we were able to divide tasks, review each other's work, and solve problems together. Through this project, we learned not only technical database skills but also the importance of communication and coordination in group work. If we had more time, we would review the logical modeling and normalization stages more carefully from the beginning. Overall, this project was a valuable experience and helped us see how database design can support a real business problem.

b. Expected Benefits of the System

We expect this system to make the maintenance process much more organized and easier to manage. Instead of relying on scattered emails, phone calls, or verbal requests, the company would have one place to keep track of work orders, technician assignments, inspections, and parts used. This would make it easier to avoid missed requests, reduce delays, and improve communication between employees, technicians, and management. The system could also help the company look back at past records, notice repeated issues, and make better decisions about maintenance planning and resource use. Overall, we believe this system would help make daily operations more efficient and more reliable.